

Modul 12 Java Servlet & JDBC Part 1

Tujuan Praktikum

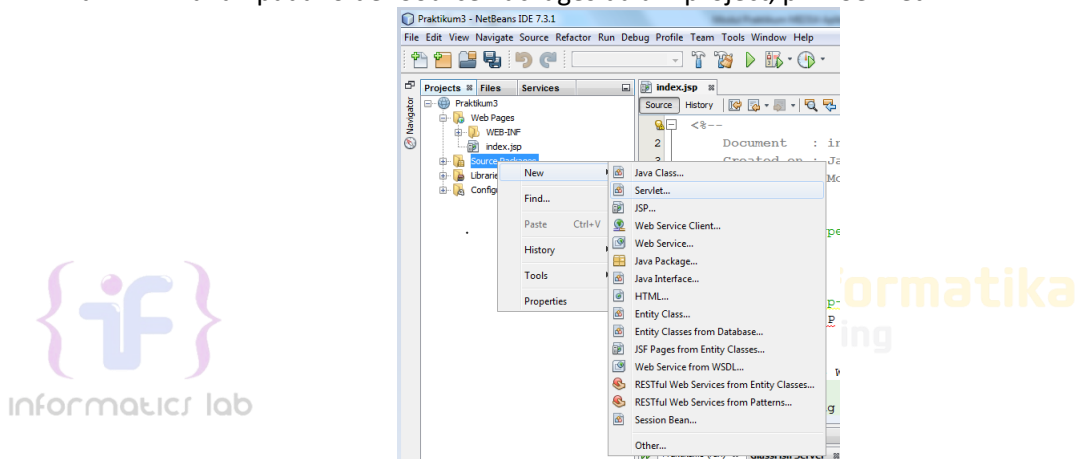
1. Mengerti konsep *Java Web* dalam Java dan dapat mengimplementasikannya dalam sebuah web sederhana.
2. Mengimplementasikan JDBC ke dalam suatu program java yang menggunakan JSP
3. Mengimplementasikan metode fetching dan create pada konsep JDBC

12.1 Java Servlet

Java Web Server memiliki Servlet engine untuk memproses konten secara dinamis. Servlet bertindak sebagai controller dalam aplikasi.

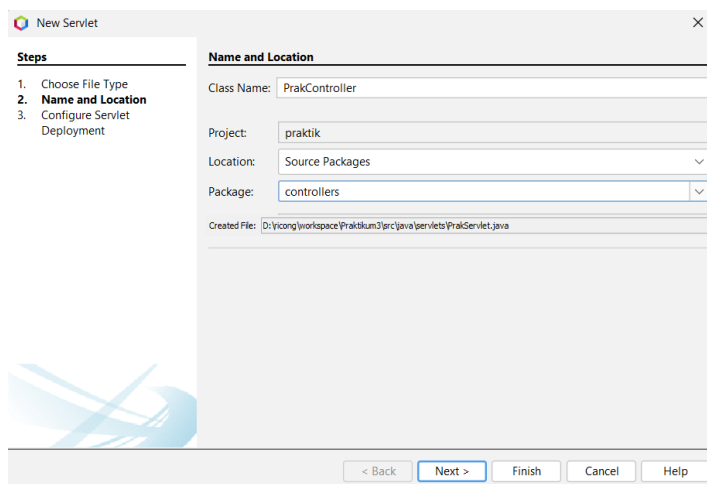
Berikut adalah contoh sederhana implementasi servlet:

- 1) Membuat servlet dalam aplikasi web
 - a. Klik kanan pada folder Source Packages dalam project, pilih Servlet



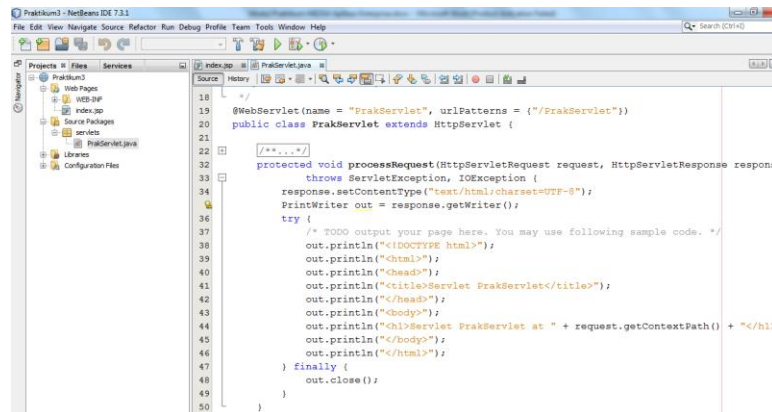
Gambar 12-1 Tampilan menu membuat Servlet

- b. Beri nama servlet 'PrakController' dan isi field *package* dengan 'controllers', lalu klik Finish



Gambar 12-2 Tampilan dialog New Servlet

c. Servlet berhasil dibuat



Gambar 12-3 Tampilan utama Servlet

- 2) Buat form pada index.jsp untuk mengirim data ke PrakController dan menampilkan hasil kembalian dari PrakController

```
<form method="get" action="PrakController">
  NIM: <input type="text" name="nama" /> <br />
  Nama: <input type="text" name="harga" /> <br />
  <input type="submit" value="Kirim" />
</form>
<%
  out.print(request.getAttribute("nama")+"<br />");
  out.print(request.getAttribute("harga")+"<br />");
%>
```

- 3) Hapus semua kode di dalam prosedur processRequest() pada Prakservlet.java dan buat kode agar dapat menerima data dari index.jsp lalu mengirim data yang telah diubah ke index.jsp

```
String nim = request.getParameter("nama");
String nama = request.getParameter("harga");
nim = "Nama Barang adalah: "+nama;
nama = "Haraga Barang adalah: "+nama;
request.setAttribute("nama", nama);
request.setAttribute("harga", harga);
request.getRequestDispatcher("index.jsp").forward(request, response);
```

12.2 Memulai JDBC

Java juga menyediakan fungsi untuk menghubungkan suatu aplikasi yang dibangun dengan menggunakan bahasa pemrograman java dengan database. Untuk menghubungkannya menggunakan JDBC. Dengan JDBC, programmer dapat menggunakan SQL statement untuk membentuk, memanipulasi atau memelihara suatu data.

Java Database Connectivity adalah versi ODBC yang dibuat oleh Sun Microsystems. Cara kerjanya mirip tapi JDBC sepenuhnya dibangun dengan java API sehingga ia dapat dipakai cross platform sedangkan ODBC dibangun dengan bahasa C sehingga ia hanya dapat dibangun pada platform spesifik. Seperti ODBC, JDBC didasarkan pada **X/Open SQL Level Interface (CLI)**.

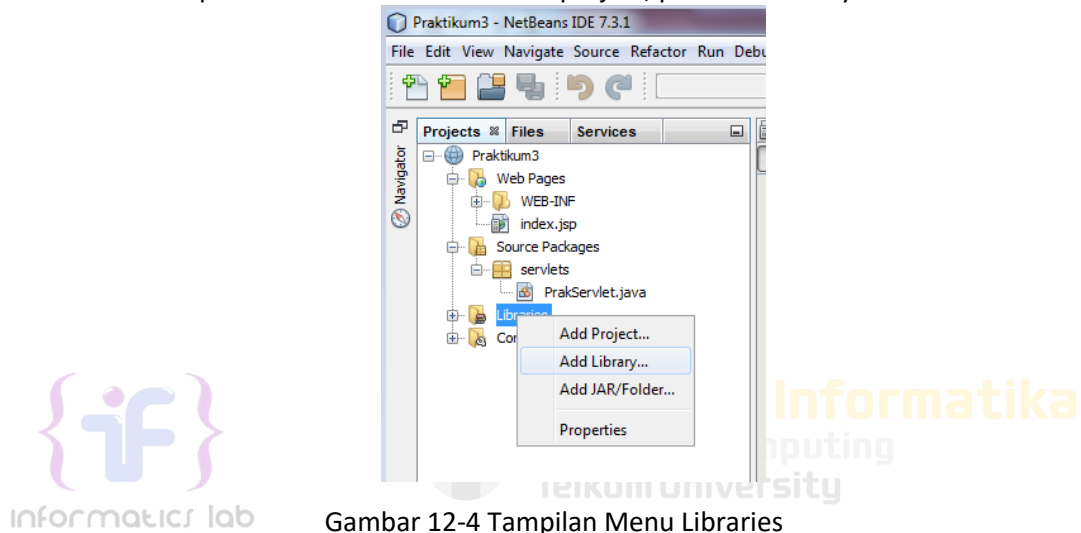
Dengan JDBC API, kita dapat mengakses database. Untuk itu, diperlukan *driver* yang akan dipakai untuk mengakses *database* di server dari dalam program kita (*client*). Setiap server dari vendor

berbeda memiliki *driver* yang berbeda. *Driver* ini dapat kita *download* dari situs Java atau dari situs vendor yang bersangkutan.

Dengan JDBC, kita dapat melakukan mengirimkan perintah-perintah SQL ke RDBMS. Kelas-kelas serta *interface* JDBC API dapat diakses dalam paket Java, sql(core API) atau javax.sql (Standard Extension API).

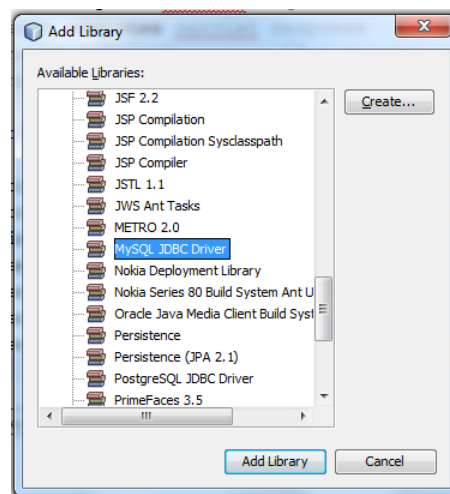
Berikut adalah contoh sederhana implementasi JDBC:

- 1) Koneksi aplikasi web dengan JDBC
 - a. Jalankan MySQL server. Pastikan database server memiliki user root dan tanpa password , dan memiliki database bernama 'test'. Dalam database 'test' buat tabel 'barang' dengan 3 kolom yaitu 'id' (int(5), primary key, auto-increment), 'nama' (varchar(20)), harga (double).
 - b. Klik kanan pada folder Libraries di dalam project, pilih Add Library



Gambar 12-4 Tampilan Menu Libraries

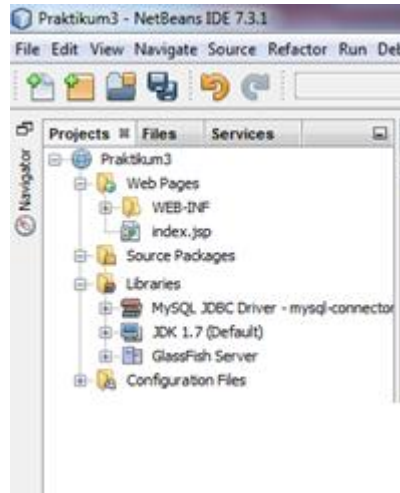
- c. Pilih MySQL JDBC Driver, klik Add Library



Gambar 12-5 Tampilan dialog Add Library

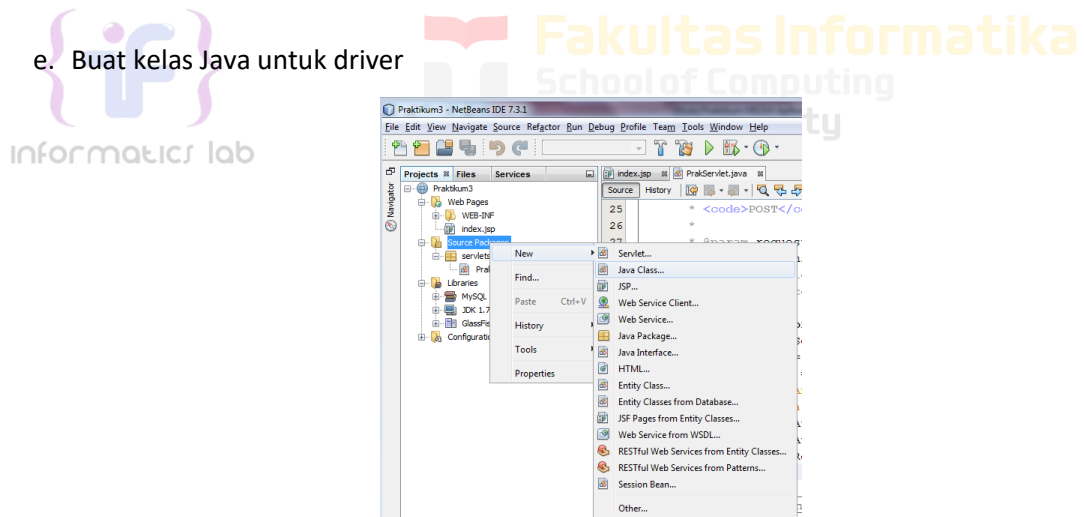
Jika MySQL JDBC Driver tidak terdapat pada Library, kita bisa menambahkan secara manual. Download di <https://dev.mysql.com/downloads/connector/j/>, pilih Platform Independent. Ekstrak file hasil download dan pindahkan ke dalam folder project. Kemudian klik kanan pada folder Libraries di dalam project, pilih Add JAR/Folder. Arahkan ke file yang sudah diekstrak tadi secara Relative Path.

d. JDBC driver untuk MySQL sudah ditambahkan



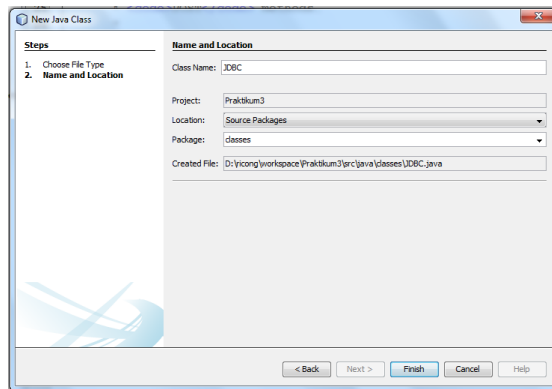
Gambar 12-6 Tampilan setelah Add Library JDBC

e. Buat kelas Java untuk driver



Gambar 12-7 Tampilan menu menambahkan Class

- f. Beri nama kelas 'JDBC', isi field package dengan 'classes'



Gambar 12-8 Tampilan dialog New Java Class

- g. Tambahkan "import java.sql.*" pada kelas JDBC.java, dan deklarasikan atribut. Buat getter untuk atribut message

```
package classes;
import java.sql.*;

public class JDBC {
    private Connection con;
    private Statement stmt;
    private boolean isConnected;
    private String message;
}
```

- h. Buat prosedur di kelas JDBC.java untuk membuka koneksi dengan database dengan parameter yang digunakan dalam koneksi database

```
public void connect() {
    String dbname = "test";
    String username = "root";
    String password = "";
    try {
        Class.forName("com.mysql.jdbc.Driver");
        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/"
+ dbname, username, password);
        stmt = con.createStatement();
        isConnected = true;
        message = "DB connected";
    } catch (Exception e) {
        isConnected = false;
        message = e.getMessage();
    }
}
```

- i. Buat prosedur untuk menutup koneksi database

```
private void disconnect() {
    try {
        stmt.close();
        con.close();
    } catch (Exception e) {
        message = e.getMessage();
    }
}
```

2) Melakukan query Sederhana

- a. Buat prosedur untuk menjalankan query database (DDL / DML) di dalam kelas JDBC.java

```
public void runQuery(String query) {
    try {
        connect();
        int result = stmt.executeUpdate(query);
        message = "info: " + result + " rows affected";
    } catch (Exception e) {
        message = e.getMessage();
    } finally {
        disconnect();
    }
}
```

- b. Buat package 'controllers' dan Servlet baru TestController di dalamnya, panggil runQuery di dalam processRequest() untuk menguji koneksi database dan melakukan query insert, output hasilnya

```
JDBC db = new JDBC();
db.runQuery("insert into barang (nama) values ('PC')");
response.setContentType("text/html;charset=UTF-8");
try (PrintWriter out = response.getWriter()) {
    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet Test</title>");
    out.println("</head>");
    out.println("<body>");
    out.println(db.getMessage() + "<br />");
    out.println("</body>");
    out.println("</html>");
}
```

12.3 Fetching Data

Berikut adalah contoh penggunaan kode untuk menghapus sebuah data yang diinginkan dari sebuah database.

- 1) Buat fungsi pada kelas JDBC.java untuk mengambil data dari database

```
public ResultSet getData(String query) {
    ResultSet rs = null;
    try {
        connect();
        rs = stmt.executeQuery(query);
    } catch (Exception e) {
        message = e.getMessage();
    }
    return rs;
}
```

- 2) Buat Servlet baru, 'BarangController' dan letakkan di dalam package 'controllers'. Ganti kode pada prosedur processRequest() menjadi kode untuk memanggil fungsi pengambilan data dari database

```
JDBC db = new JDBC();
ResultSet rs = db.getData("select * from barang");
request.setAttribute("list", rs);
request.getRequestDispatcher("view.jsp").forward(request, response);
```

- 3) Buat halaman JSP dengan nama 'view'. Tambahkan JSP directive diatasnya untuk mengimpor kelas ResultSet agar dapat digunakan pada halaman

```
<%@page import="java.sql.ResultSet"%>
```

- 4) Buat scriptlet pada halaman view.jsp untuk menerima data dari Servlet, dan tampilkan dalam bentuk tabel

```
<h2>Daftar Barang</h2>
<table border="1" cellpadding="5" cellspacing="0">
  <tr>
    <th>ID</th>
    <th>Nama</th>
    <th>Harga</th>
  </tr>
  <%
    ResultSet rs = (ResultSet) request.getAttribute("list");
    if (rs != null) {
      while (rs.next()) {
        int id = rs.getInt("id");
        String nama = rs.getString("nama");
        double harga = rs.getDouble("harga");
      %>
      <tr>
        <td><%= id %></td>
        <td><%= nama %></td>
        <td><%= harga %></td>
      </tr>
    }
  <%>
</table>
<br>
<a href="form.jsp">Tambah Barang</a>
```

Dengan menggunakan kode di atas, saat halaman BarangController diakses, program akan mengambil data dari tabel barang dalam database. Data yang diambil kemudian ditampilkan dalam bentuk tabel di halaman tersebut, menampilkan kolom ID, Nama dan Harga untuk setiap barang yang ada di database.

12.4 Insert Data

Berikut adalah contoh penggunaan kode untuk *insert* sebuah data yang diinginkan ke database.

- 1) Buka servlet 'BarangController', ganti dan tambahkan kode pada prosedur processRequest() menjadi kode untuk menerima parameter dari URL BarangController. Tambahkan aksi untuk menambah data ke database, lalu kembali ke daftar barang

```
JDBC db = new JDBC();
String menu = request.getParameter("menu");
if (request.getParameterMap().isEmpty()) {
  ResultSet rs = db.getData("select * from barang");
  request.setAttribute("list", rs);
  request.getRequestDispatcher("view.jsp").forward(request, response);
} else if ("add".equals(menu)) {
  request.getRequestDispatcher("form.jsp").forward(request, response);
} else if ("insert".equals(menu)) {
  String nama = request.getParameter("nama");
  Double harga = Double.parseDouble(request.getParameter("harga"));
```

```
db.runQuery("INSERT INTO barang (nama, harga) VALUES ('" + nama +
"', " + harga+ ")"); response.sendRedirect("BarangController");
}
```

- 2) Buat halaman JSP 'form.jsp', dan isi dengan form penambahan barang

```
<form method="post" action="BarangController?menu=insert">
  Nama Barang: <input type="text" name="nama" /><br />
  Harga Barang: <input type="text" name="harga" /><br />
  <input type="submit" value="Tambah" />
</form>
```

- 3) Uji dengan membuka URL .../BarangController?menu=add

